CYBERSECURITY LAB PROJECT

Wazuh SIEM Monitoring & Detection

Expanded Project Analysis: Windows 11 Pro + Kali Linux

Project element	Recorded lab details
SIEM platform	Wazuh 4.14.8
Central Wazuh Manager	192.168.0.142
Windows endpoint	Windows 11 Pro · Windows11-Lab-01 · Agent ID 004 · 192.168.0.100
Windows telemetry	Windows Event Logs + Microsoft Sysmon Operational channel
Kali endpoint	cyb3rgwd · Agent ID 005 · 192.168.0.193
Kali telemetry	Wazuh Syscheck / File Integrity Monitoring (FIM) + Linux logs
Archive index	wazuh-archives-*
Status in source report	Windows and Kali phases validated; Metasploitable 3 planned next

Purpose of this document

This report explains the recorded experiment in presentation-ready language: the security problem, architecture, implementation sequence, tests, observations, interpretation, limitations, troubleshooting, and a short oral-defence guide. It distinguishes verified results in the supplied report from illustrative commands that may be useful for reproducing or explaining the lab.

Prepared from the supplied nine-page combined project report. Commands shown as examples are not claimed to be exact transcripts of every command used during the original session.

1. Executive Summary

The project built and validated a small, multi-platform security monitoring laboratory. Wazuh acted as the central security information and event management (SIEM) platform. A Windows 11 Pro endpoint reported Windows event data and Microsoft Sysmon telemetry; a Kali Linux endpoint reported file-integrity events. Both agents connected to the same Wazuh Manager at 192.168.0.142.

On Windows, controlled execution of programs such as `whoami.exe` and Notepad generated process telemetry. The report records Sysmon Event ID 1 (process creation) and Event ID 5 (process termination) observations, and an investigation of a Wazuh Account Discovery detection mapped to MITRE ATT&CK technique T1087. Archive logging and indexing were enabled so raw events could be searched in Discover.

On Kali, a test file at `/wazuh-test/fim-test-02.txt` was created, modified, and deleted. Wazuh detected the three changes. The modification event recorded changes in file size, modification time, MD5, SHA1, and SHA256 values. These controlled events demonstrate that telemetry traversed the endpoint-to-manager pipeline; they do not, by themselves, demonstrate that either endpoint was compromised.

Overall conclusion: the documented Windows and Kali monitoring phases were validated. The next planned phase in the source report is onboarding Metasploitable 3 for additional controlled security telemetry and detection exercises.

2. Problem Statement and Rationale

A computer can generate many events, but those events are not useful to a security team if they remain isolated on individual machines or cannot be searched centrally. Analysts need visibility into process execution, file changes, account-discovery activity, and related context. This project tests whether a central SIEM can receive, retain, and expose this telemetry for investigation across Windows and Linux.

The experiment focuses on defensive monitoring. It uses intentionally benign actions in a lab to check whether the monitoring pipeline notices relevant activity and preserves useful fields for analysis.

3. Key Technologies and Concepts

Term	Meaning in this experiment
SIEM	A platform that centralizes security event collection, analysis, alerting, and investigation.
Wazuh Manager	The central server that receives agent data, evaluates rules, and supports alert/archive workflows.
Wazuh Agent	Endpoint software that collects configured local telemetry and sends it to the manager.
Microsoft Sysmon	A Windows system service that records detailed activity such as process creation and termination.
Event ID 1	Sysmon process-creation event. Useful fields can include executable image, user, process ID, parent process, and hashes.
Event ID 5	Sysmon process-termination event.
FIM / Syscheck	File Integrity Monitoring detects changes to monitored files and records details about the change.
Hash/checksum	A computed digest used to help compare file content or integrity state; a changed hash can indicate content changed.
Alert vs archive	An alert is a rule-matched security finding; an archive is raw event data retained for deeper searches.
MITRE ATT&CK; T1087	Account Discovery: activity associated with identifying accounts. It can be legitimate administration or suspicious reconnaissance depending on context.
Discover / DQL	The dashboard search interface and its query syntax. DQL belongs in the dashboard search bar, not a Linux shell.

4. Lab Architecture and Data Flow

The verified architecture is a central-manager model. The endpoints collect their own telemetry; the Wazuh agents forward it to the manager; Wazuh processes the data and makes alerts or archived records available through the dashboard.

Windows data path

Windows 11 Pro → Sysmon + Windows event channels → Wazuh Agent 004 → Wazuh Manager (192.168.0.142) → rules/alerts and archives → Discover.

Kali data path

Kali Linux → Syscheck/FIM + Linux logs → Wazuh Agent 005 → Wazuh Manager (192.168.0.142) → event analysis/dashboard → FIM event investigation.

Layer	Windows 11	Kali Linux
Agent identity	Windows11-Lab-01 / ID 004	cyb3rgwd / ID 005
IP address	192.168.0.100	192.168.0.193
Validated telemetry	Windows events + Sysmon	FIM add / modify / delete
Central manager	192.168.0.142	192.168.0.142
Investigation	Alerts and raw archives in Discover	Syscheck events and file integrity fields

Why centralization matters

Centralization makes it possible to search across hosts, correlate activity, compare event context, and keep a consistent investigation workflow. In a production environment, this would support faster triage and incident response, subject to correct configuration, adequate retention, and alert tuning.

5. Step-by-Step Methodology: Windows 11 + Sysmon

Step 1 — Prepare and identify the endpoint

The endpoint was Windows 11 Pro with address 192.168.0.100. The manager address was 192.168.0.142. Confirming these identities first is important because an agent that points at an incorrect manager address cannot reliably enrol or send telemetry to the intended server.

Step 2 — Enrol the Windows agent

The source report records an initial invalid manager address of 0.0.0.0. It was corrected to 192.168.0.142. The first agent name was rejected because it was not unique, so the unique name Windows11-Lab-01 was used. Enrollment settings included port 1515, the agent name, and the default group. After the agent service was restarted, the endpoint connected successfully.

Interpretation: the manager address identifies where the agent should connect; a unique agent name avoids identity conflicts; and service restart applies the corrected configuration. The report later confirms Agent ID 004 was ACTIVE.

Step 3 — Verify the agent in the dashboard

The dashboard showed Agent 004 as ACTIVE, version 4.14.8, IP 192.168.0.100, and Windows 11 Pro. Application, Security, and System event monitoring was confirmed. This check is more meaningful than simply seeing the software installed: it verifies the central platform recognizes the endpoint as connected.

Step 4 — Install and verify Sysmon

Sysmon was downloaded from Microsoft Sysinternals, extracted, and installed. The Sysmon64 service was verified as running. Local EventChannel checks confirmed that Sysmon was generating process telemetry. Sysmon adds process-level context that may not be present in basic event logs, depending on the event source and configuration.

Step 5 — Configure Wazuh to collect Sysmon events

The Wazuh agent configuration was extended with a localfile entry for the Microsoft-Windows-Sysmon/Operational event channel using the eventchannel log format. The Wazuh service was restarted, and its log confirmed analysis of that channel. This step links local event generation to central collection.

Illustrative configuration pattern (check the installed configuration before changing it):

```
<localfile>\n  <location>Microsoft-Windows-Sysmon/Operational</location>\n  <log_format>eventchannel</log_format>\
```

Step 6 — Generate benign process activity

Controlled execution of whoami.exe and Notepad was used to create predictable process events. The report records whoami.exe at C:\\Windows\\System32\\whoami.exe, process ID 36000, user CYB3RGWD\\PC, High integrity level, and PowerShell as the parent process (PID 44144). Notepad was launched using Start-Process notepad.exe.

An illustrative PowerShell test is:

```
whoami\nStart-Process notepad.exe
```

These commands are benign test actions. They help an analyst identify what normal process creation looks like in the telemetry. Do not interpret a process name alone as proof of maliciousness.

6. Windows Detection and Investigation

Step 7 — Understand Sysmon Event ID 1 and Event ID 5

Event ID 1 records a process being created. Relevant fields can include the executable path (Image), process ID, user, parent image, parent process ID, integrity level, and hashes. Event ID 5 records process termination. Together, these event types can help reconstruct what ran and when it ended, although they do not automatically explain the intent of the process.

The source report confirms Event ID 1 and Event ID 5 were observed and archived. It specifically lists a whoami.exe process-creation event and confirms Notepad telemetry locally and in the server archives.

Step 8 — Investigate the Account Discovery detection

The report records a Wazuh detection named “Discovery activity executed” associated with the command net user. Wazuh mapped the event to MITRE ATT&CK; T1087 — Account Discovery, under the Discovery tactic. The command can enumerate local or domain account information depending on its use and environment; administrators may also use it for legitimate troubleshooting.

The process ancestry was described approximately as wazuh-agent.exe → net.exe → net1.exe → net user. The parent was wazuh-agent.exe running as SYSTEM, and the working directory was the Wazuh agent directory. Those details were consistent with Wazuh-generated or legitimate lab telemetry in this experiment. The key lesson is to examine process ancestry, account context, executable path, command line, time, and surrounding events before classifying an alert.

Analyst workflow for a detection

- Read the alert title, rule description, severity/level, timestamp, and affected agent.
- Inspect the command line and process fields; identify the user and parent process.
- Review the parent-child chain and executable paths for context.
- Compare the activity with the known lab action and expected system behaviour.
- Decide whether the event is expected, suspicious, or inconclusive; document the evidence and rationale.

Step 9 — Enable raw JSON archives

The report states that archive logging was enabled on the manager using the global logall and logall_json settings. The configuration test succeeded and the manager restarted successfully. Archive indexing was then enabled so wazuh-archives-* could be selected in Discover. More than 54,000 events appeared for the observed seven-day period. That number demonstrates substantial indexed data in the observed window, not that all events were necessarily from this one endpoint.

Why alerts and archives are different

Rules generate alerts when events match detection logic. Raw archives help analysts search event records that may not have triggered an alert. Archives are useful for hunting and validation, but enabling them increases storage use and may capture sensitive information; retention and access should be managed carefully.

7. Step-by-Step Methodology: Kali Linux FIM

Step 10 — Enrol Kali as a second endpoint

Kali Linux was connected to the same manager at 192.168.0.142. The agent was named cyb3rgwd, assigned Agent ID 005, and recorded at 192.168.0.193. The source report says the Wazuh agent service was active/running. This established a second endpoint for multi-platform monitoring.

Step 11 — Choose a controlled test file

The experiment used /wazuh-test/fim-test-02.txt. A dedicated test path is useful because it makes the change intentional, easy to locate, and less likely to affect real system files. FIM only detects the changes it is configured to monitor; the report does not provide the complete syscheck configuration, so do not assume every file on the Kali machine was monitored.

Step 12 — Create the file

A new file was created at the monitored path. Wazuh reported the event as added. This validates the initial detection path: a filesystem change occurs, the agent notices it, and the event becomes visible in the dashboard.

Step 13 — Modify the file

New content was written to the same file. Wazuh reported a modified event, with the decoder syscheck_integrity_changed and description Integrity checksum changed. The event included changes to size, modification time, MD5, SHA1, and SHA256 values, and was processed in real-time mode.

Illustrative safe test sequence (run only in your lab and ensure the directory is appropriate):

```
sudo mkdir -p /wazuh-test\nprintf "initial test content\\n" | sudo tee /wazuh-test/fim-test-02.txt\nprintf "update
```

Step 14 — Delete the file

The test file was removed. Wazuh reported a deleted event, with decoder syscheck_deleted and rule description File deleted; the report shows rule level 5. This verifies that the monitored path generated a deletion record as well as add and modify events.

```
sudo rm /wazuh-test/fim-test-02.txt
```

Only remove the dedicated test file created for this experiment. Never use a broad deletion command on an unknown path.

8. How to Investigate the Kali Events

The report describes a dashboard query for the modification event:

```
agent.name:cyb3rgwd AND rule.groups:syscheck AND syscheck.event:modified
```

It also describes filtering for syscheck.event:deleted to locate the deletion event. These are dashboard-side DQL searches. A query pasted into the Kali terminal is interpreted as a shell command, which explains the reported “command not found” message. That was a command-context mistake, not evidence of a broken Wazuh agent.

When opening an event, verify these fields where available:

- Agent name and endpoint identity — confirms which monitored host produced the event.
- syscheck.event — identifies whether the event was added, modified, or deleted.
- syscheck.path — confirms the affected file path.
- Rule description and level — explains how Wazuh classified the event.
- Integrity fields — compare file size, modification time, and hashes when present.
- Timestamp — places the change in the incident timeline.

A changed checksum indicates that the file's integrity state changed. It does not prove malicious tampering on its own: normal editing, software updates, or an intentional lab test can produce the same signal. The analyst must compare the change with the expected activity and surrounding evidence.

9. Results and Evidence Matrix

Test / component	Observed result in supplied report	What it demonstrates
Wazuh Manager	Active/running	Central platform was available.
Windows Agent 004	ACTIVE; 192.168.0.100	Windows endpoint was enrolled and connected.
Sysmon64	RUNNING	Sysmon service was operational.
Sysmon channel	Collected by agent	Windows process telemetry was configured for collection.
Event ID 1 / 5	Observed and archived	Process creation/termination telemetry reached the archive path.
Account Discovery	Observed; MITRE T1087 mapping	Detection mapping and contextual investigation were practised.
Kali Agent 005	Service active; endpoint 192.168.0.193	Second operating system was connected.
FIM file creation	Added event	New-file detection worked for the test path.
FIM file modification	Modified; checksum change	File integrity metadata was recorded.
FIM file deletion	Deleted; rule level 5	Deletion detection worked for the test path.
Discover archives	wazuh-archives-* available; >54,000 events in observed seven-day period	Raw event search and indexing were available.

Evidence presentation checklist

- Show the Wazuh Agents page with Windows Agent 004 and Kali Agent 005 active.
- Show a Windows Sysmon Event ID 1 record with process image, user, and parent process.
- Show the Account Discovery detection and explain why the process ancestry mattered.
- Show the wazuh-archives-* data view in Discover.

- Show the Kali modified event with checksum-related fields.
- Show the Kali deleted event for the same test path.

10. Troubleshooting and Lessons Learned

Issue	Cause / interpretation	Lesson
Manager address set to 0.0.0.0	The configured address was not the intended Wazuh manager.	Use the correct manager IP and revalidate agent status.
Agent name rejected	The name was not unique.	Use a unique descriptive name per endpoint.
DQL entered in Kali shell	Dashboard query syntax is not a shell command.	Run DQL in the Wazuh dashboard search bar.
No event visible immediately	Could involve collection settings, monitored path, indexing, time filters, or query syntax.	Check service status, agent logs, channel/path configuration, time range, and index selection systematically.

A good troubleshooting method changes one variable at a time and checks the result after each change. Record the original symptom, the change made, the service restarted if applicable, and the evidence that confirms the fix.

11. Security Interpretation, Scope, and Limitations

What the experiment proves

- The Windows agent was connected and Windows/Sysmon telemetry was collected.
- The Wazuh dashboard showed relevant process and Account Discovery events.
- Raw archive logging/indexing was available for investigation in Discover.
- The Kali agent generated FIM events for controlled file creation, modification, and deletion.
- The two endpoint phases used the same central Wazuh Manager, demonstrating a multi-platform monitoring pattern.

What it does not prove

- It does not prove the endpoints were compromised; the test actions were deliberately benign.
- It does not prove every possible Windows or Linux event was collected.
- It does not establish alert precision, false-positive rate, detection coverage, or response time across a representative attack set.
- It does not demonstrate production hardening, high availability, long-term retention, or a complete incident-response process.
- The report does not include the full agent configuration, complete command transcript, or all raw event JSON; avoid claiming details that are not documented.

Ethical and operational controls

Keep testing within systems you own or are authorized to assess. Use isolated virtual machines and dedicated test files. Avoid destructive commands and avoid generating real malicious activity on a production network. If archives are enabled, consider storage requirements, retention, sensitive data exposure, and access controls.

12. Recommendations and Next Phase

- Continue with the planned Metasploitable 3 integration only in the isolated lab network and with explicit authorization.
- Record the new VM's IP address, agent identity (if an agent is supported and installed), and connectivity tests before generating activity.
- Define a small set of controlled tests, expected telemetry, and success criteria before each exercise.
- Keep a screenshot or exported event for every test, including the event timestamp, agent, rule, and relevant fields.
- Tune rules only after documenting baseline behaviour; do not suppress an alert solely because its title looks alarming or familiar.
- Review archive retention and storage growth if the lab is left running for extended periods.
- Maintain a change log with configuration edits, service restarts, errors, fixes, and validation results.

13. Conclusion

The combined lab demonstrates a working defensive monitoring workflow across Windows and Linux. Windows telemetry was enriched by Sysmon, forwarded through Wazuh Agent 004, and examined using detections and raw archives. Kali Linux was onboarded as Agent 005, and its FIM capability detected the controlled addition, modification, and deletion of a test file. The most important analytical lesson is that an alert or checksum change is a lead for investigation—not a verdict. Process ancestry, identity, file path, hashes, timestamps, and known lab activity must be considered together.

The source report marks both Windows and Kali phases as completed/validated. Metasploitable 3 is listed as the next planned phase, not as a completed phase in this document.

14. Short Presentation Script

“My project is a multi-platform endpoint monitoring lab using Wazuh 4.14.8. The central manager is 192.168.0.142. I first connected a Windows 11 Pro endpoint and used Microsoft Sysmon to collect detailed process telemetry. I generated benign process activity, verified Sysmon Event ID 1 and Event ID 5, investigated an Account Discovery detection mapped to MITRE ATT&CK T1087, and enabled raw archive searching in Discover. I then connected Kali Linux as a second endpoint and tested File Integrity Monitoring by creating, modifying, and deleting a dedicated test file. Wazuh detected all three operations, and the modification event included checksum and file metadata changes. These results validate the collection and investigation pipeline in the lab; they do not indicate that the test machines were compromised. My next planned phase is to integrate Metasploitable 3 for further controlled detection exercises.”

15. Questions You May Be Asked

Question	Suggested answer
What is Wazuh?	An open-source security monitoring platform used here for endpoint telemetry, rule-based detections, FIM, and centralized investigation.
Why use Sysmon on Windows?	It provides detailed system activity records, including process creation and termination, which help analysts investigate execution context.
What is the difference between an alert and an archive?	An alert is generated by detection logic; an archive preserves raw events for broader searching and hunting.
What does T1087 mean?	MITRE ATT&CK; Account Discovery. The activity may be legitimate or suspicious depending on context.
Why investigate parent processes?	Parent-child relationships help explain how a command started and whether it fits expected activity.
What does FIM detect?	Changes to monitored files, such as additions, modifications, and deletions.
Does a changed hash prove an attack?	No. It proves the file's integrity state changed; intent must be assessed using context and supporting evidence.
Why was DQL rejected in the Kali terminal?	DQL is a dashboard query language, not a Linux shell command.
What is the main limitation?	The tests validate selected telemetry paths, not all detection scenarios or production readiness.
What is next?	The source report plans to onboard Metasploitable 3 and run controlled monitoring/detection exercises.

Reference basis: supplied file, “Cyb3rgwd_Wazuh_Windows_Kali_Combined_Project_Analysis.pdf,” pages 1–9. All endpoint identities, addresses, observed test results, and status statements above are taken from that report. Any sample commands are marked illustrative and should be checked against the actual lab configuration before use.